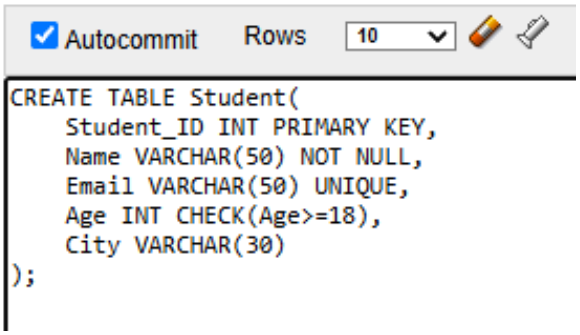
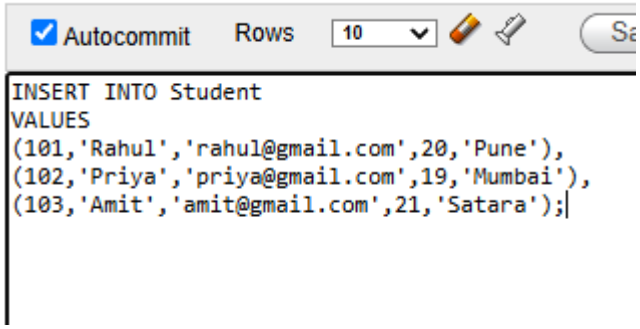
Assignment No.1

Aim: To create simple tables with Primary Key (Field Level and Table Level) and implement Primary Key, Foreign Key, CHECK, UNIQUE, and NOT NULL constraints.:

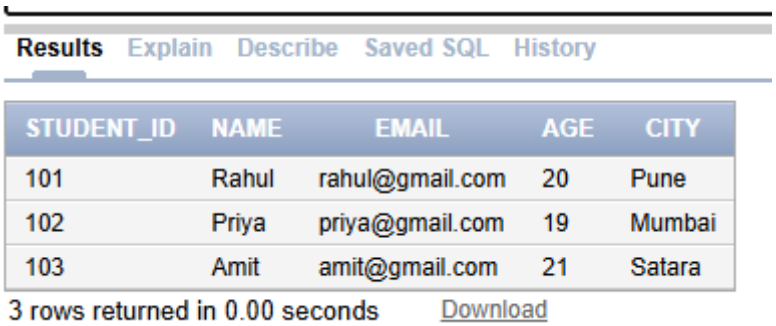
Step 1:-Create table student



Step 2: Insert Records



Step 3: Display Table
SELECT * FROM Student;



Step 4: Create Department Table (Table Level Primary Key)

```
CREATE TABLE Employee(
    Emp_ID INT,
    Emp_Name VARCHAR(50) NOT NULL,
    Salary DECIMAL(10,2) CHECK(Salary>=10000),
    Email VARCHAR(50) UNIQUE,
    Dept_ID INT,
    CONSTRAINT PK_Employee PRIMARY KEY(Emp_ID),
    CONSTRAINT FK_Department FOREIGN KEY(Dept_ID)
    REFERENCES Department(Dept_ID)
);
```

Step 5: Insert Records

Autocommit Rows 10

```
INSERT INTO Department
VALUES
(1,'Computer','Pune'),
(2,'Mechanical','Kolhapur'),
(3,'Civil','Satara');
```

Step 6: Display Department Table

Results Explain Describe Saved SQL History

DEPT_ID	DEPT_NAME	LOCATION
1	Computer	Pune
2	Mechanical	Kolhapur
3	Civil	Satara

3 rows returned in 0.00 seconds Download

Step 7: Create Employee Table with Constraints

```
CREATE TABLE Employee(
    Emp_ID INT,
    Emp_Name VARCHAR(50) NOT NULL,
    Salary DECIMAL(10,2) CHECK(Salary>=10000),
    Email VARCHAR(50) UNIQUE,
    Dept_ID INT,
    CONSTRAINT PK_Employee PRIMARY KEY(Emp_ID),
    CONSTRAINT FK_Department FOREIGN KEY(Dept_ID)
    REFERENCES Department(Dept_ID)
);
```

Step 8: Insert Records

Autocommit Rows 10

```
INSERT INTO Employee
VALUES
(1,'Rohit',35000,'rohit@gmail.com',1),
(2,'Sneha',40000,'sneha@gmail.com',2),
(3,'Ajay',28000,'ajay@gmail.com',3);
```

Step 11: Display Employee Table

Results Explain Describe Saved SQL History

EMP_ID	EMP_NAME	SALARY	EMAIL	DEPT_ID
1	Rohit	35000	rohit@gmail.com	1
2	Sneha	40000	sneha@gmail.com	2
3	Ajay	28000	ajay@gmail.com	3

3 rows returned in 0.00 seconds Download

Output:-

Student Table:-

Autocommit Rows 10 Save Run

```
DESC Student;
```

Results Explain Describe Saved SQL History

Object Type TABLE Object STUDENT

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
STUDENT	STUDENT_ID	NUMBER	22	-	0	1	-	-	-
	NAME	VARCHAR2	50	-	-	-	-	-	-
	EMAIL	VARCHAR2	50	-	-	-	✓	-	-
	AGE	NUMBER	22	-	0	-	✓	-	-
	CITY	VARCHAR2	30	-	-	-	✓	-	-

1 - 5

Department Table:-

Autocommit Rows 10 Save Run

```
DESC Department;
```

Results Explain Describe Saved SQL History

Object Type TABLE Object DEPARTMENT

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DEPARTMENT	DEPT_ID	NUMBER	22	-	0	1	-	-	-
	DEPT_NAME	VARCHAR2	50	-	-	-	✓	-	-
	LOCATION	VARCHAR2	40	-	-	-	✓	-	-

1 - 3

Employee Table:-

Autocommit Rows 10 Save Run

```
DESC Employee;
```

Results Explain Describe Saved SQL History

Object Type TABLE Object EMPLOYEE

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
EMPLOYEE	EMP_ID	NUMBER	22	-	0	1	-	-	-
	EMP_NAME	VARCHAR2	50	-	-	-	-	-	-
	SALARY	NUMBER	-	10	2	-	✓	-	-
	EMAIL	VARCHAR2	50	-	-	-	✓	-	-
	DEPT_ID	NUMBER	22	-	0	-	✓	-	-

1 - 5

Assignment No.2

Aim:- To drop a table, alter schema of a table, insert / update / delete records using tables created in previous Assignments. (Use simple forms of insert / update /delete statements).

1)Insert

2)Update

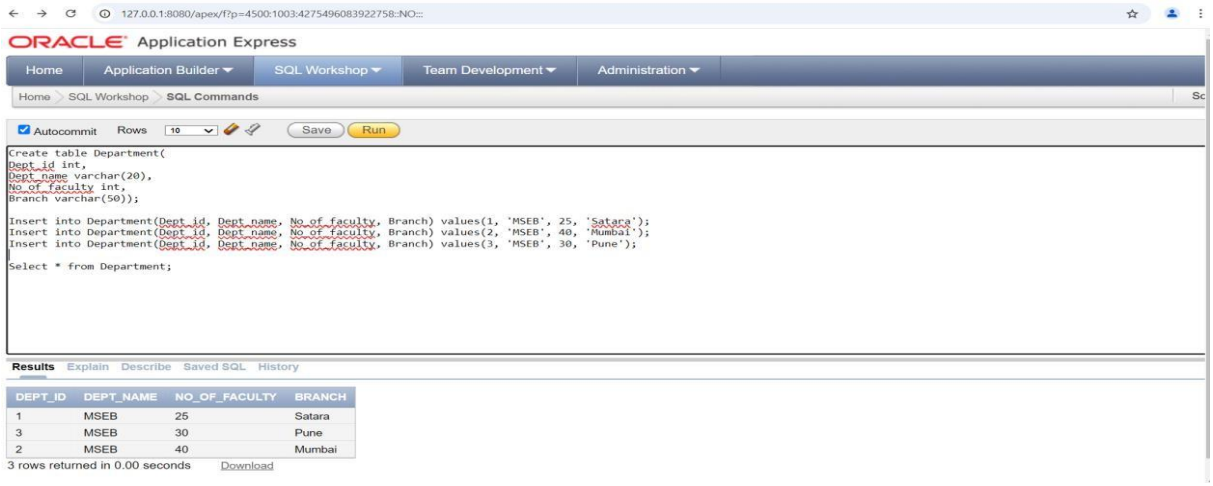
3)Delete

Input:-

```
Create table Department(
Dept_id int,
Dept_name varchar(20),
No_of_faculty int,
Branch varchar(50));

Insert into Department(Dept_id, Dept_name, No_of_faculty, Branch) values(1, 'MSEB', 25, 'Satara');
Insert into Department(Dept_id, Dept_name, No_of_faculty, Branch) values(2, 'MSEB', 40, 'Mumbai');
Insert into Department(Dept_id, Dept_name, No_of_faculty, Branch) values(3, 'MSEB', 30, 'Pune');
```

Output:-

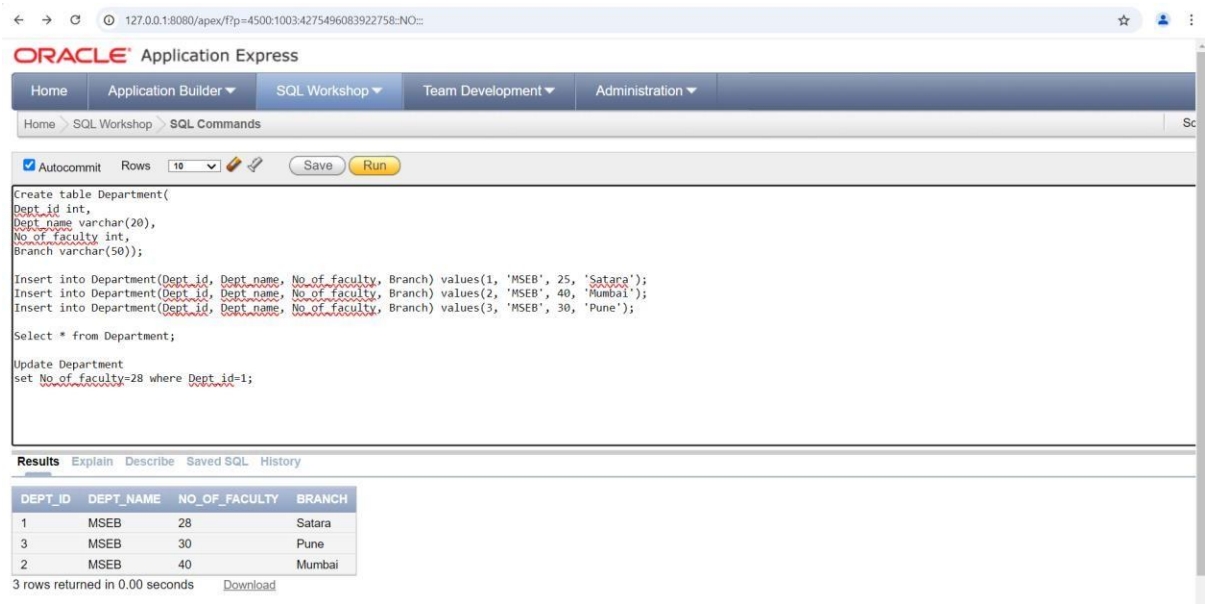


Input:-

Update Department

set No_of_faculty=28 where Dept_id=1;

Output:-

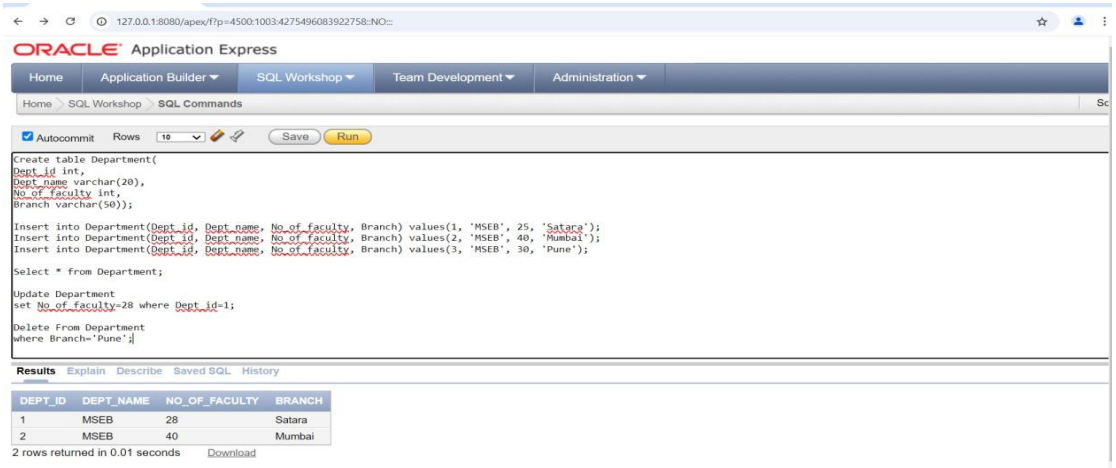


Input:-

Delete From Department

where Branch='Pune';

Output:-



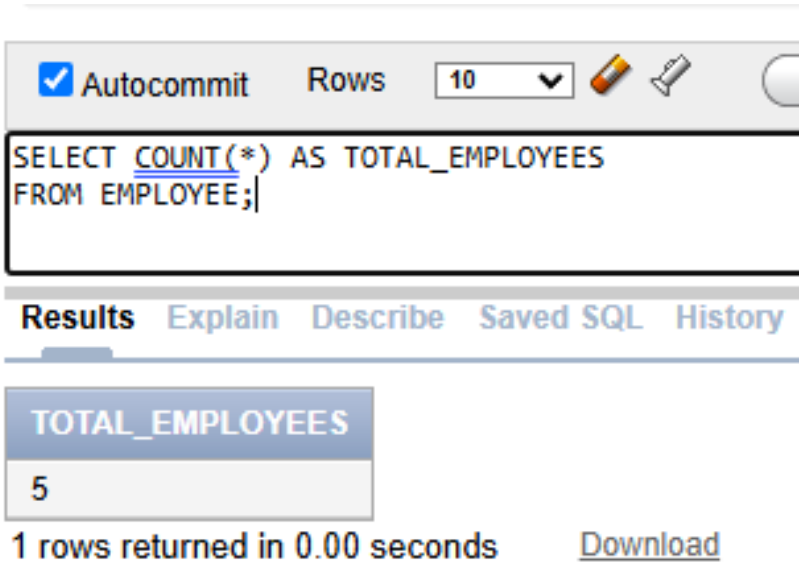
Assignment No. 3

Aim: Write the query for implementing the following functions:

1.Count( )

```
SELECT COUNT(*) AS TOTAL_EMPLOYEES
FROM EMPLOYEE;
```

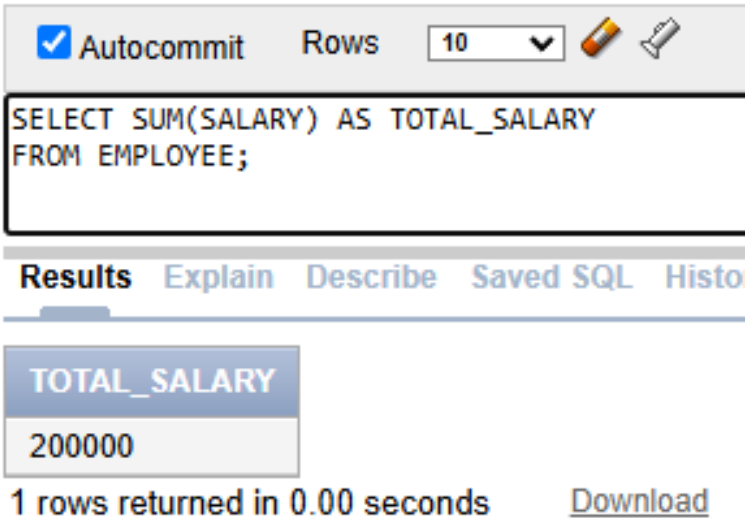
Output:



2.SUM( )

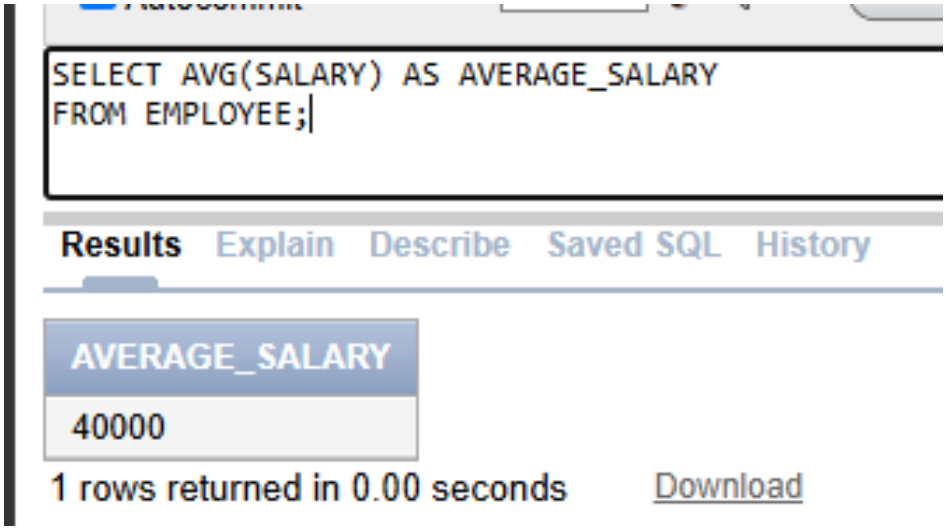
```
SELECT SUM(SALARY) AS TOTAL_SALARY
FROM EMPLOYEE;
```

Output:



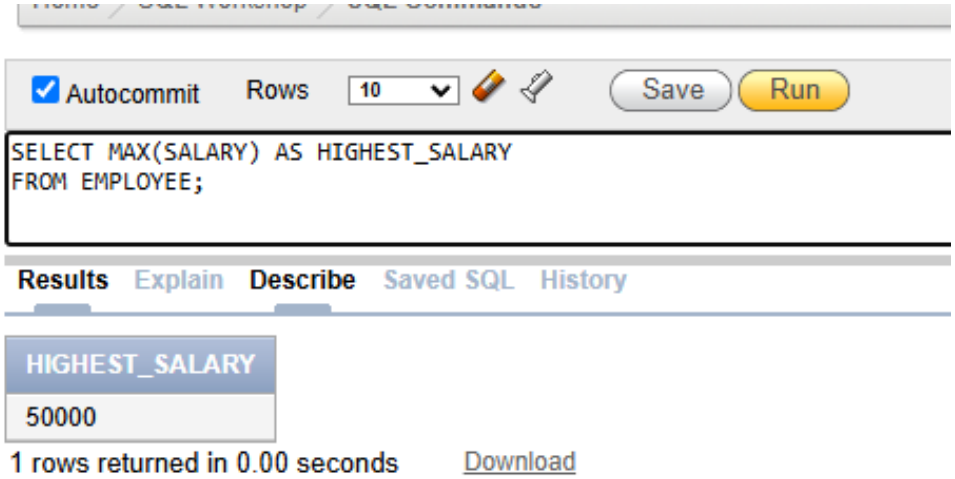
3.AVG( )

SELECT AVG(SALARY) AS AVERAGE_SALARY
FROM EMPLOYEE;



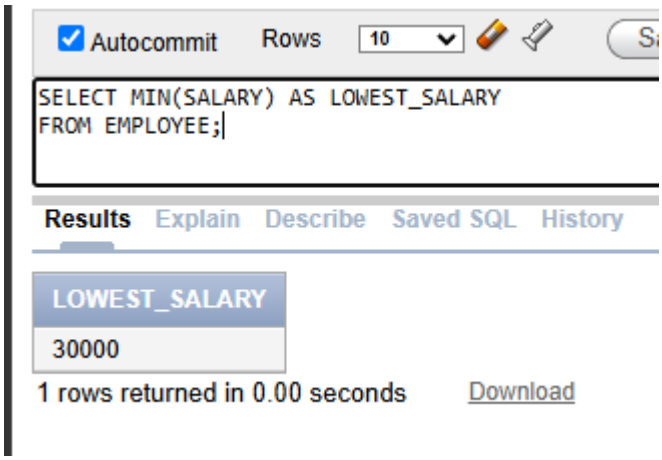
3.MAX( )

Output: SELECT MAX(SALARY) AS HIGHEST_SALARY



4.MIN( )

SELECT MIN(SALARY) AS LOWEST_SALARY
FROM EMPLOYEE;



Assignment No. 4

Aim:- Write the queries to implement the joins.

1) Left join

2) Right join

3)Full join

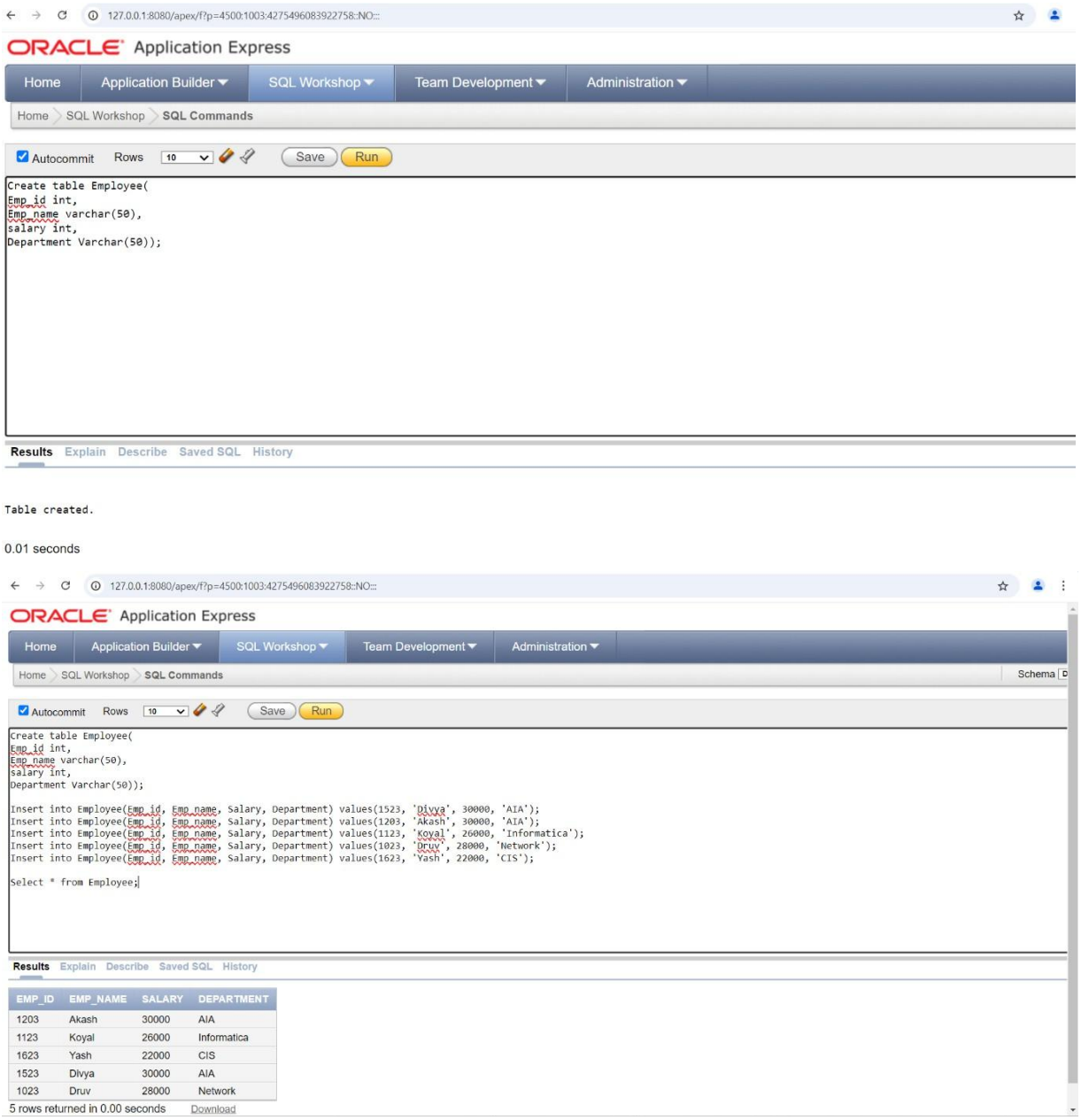
Input:-

```
Create table Employee(
Emp_id int,
Emp_name varchar(50),
salary int,
Department Varchar(50));

Insert into Employee(Emp_id, Emp_name, Salary, Department) values(1523, 'Divya', 30000, 'AIA');
Insert into Employee(Emp_id, Emp_name, Salary, Department) values(1203, 'Akash', 30000, 'AIA');
Insert into Employee(Emp_id, Emp_name, Salary, Department) values(1123, 'Koyal', 26000, 'Informatica');
Insert into Employee(Emp_id, Emp_name, Salary, Department) values(1023, 'Druv', 28000, 'Network');
Insert into Employee(Emp_id, Emp_name, Salary, Department) values(1623, 'Yash', 22000, 'CIS');

Select * from Employee;
```

Output:-



Input:-

Create table Project(

Project_id int,

Emp_id int,

Department varchar(50),

Name_of_project Varchar(50));

Insert into Project(Project_id, Emp_id, Department, Name_of_project) values(1, 1203, 'AIA', 'Pearson');

Insert into Project(Project_id, Emp_id, Department, Name_of_project) values(2, 1523, 'AIA', 'Pearson');

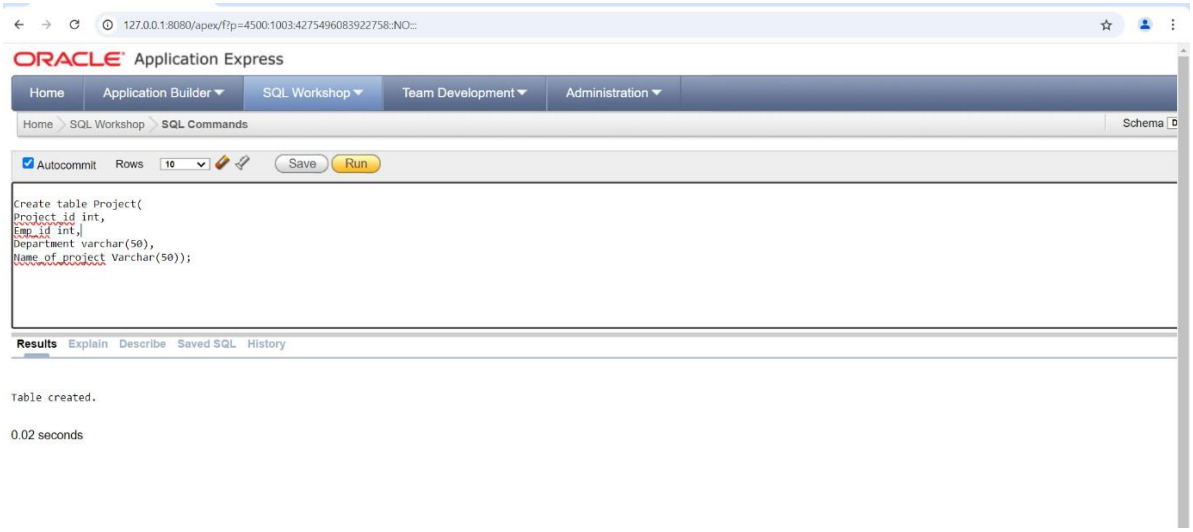
Insert into Project(Project_id, Emp_id, Department, Name_of_project) values(3, 1123, 'Informatica', 'AR and VR');

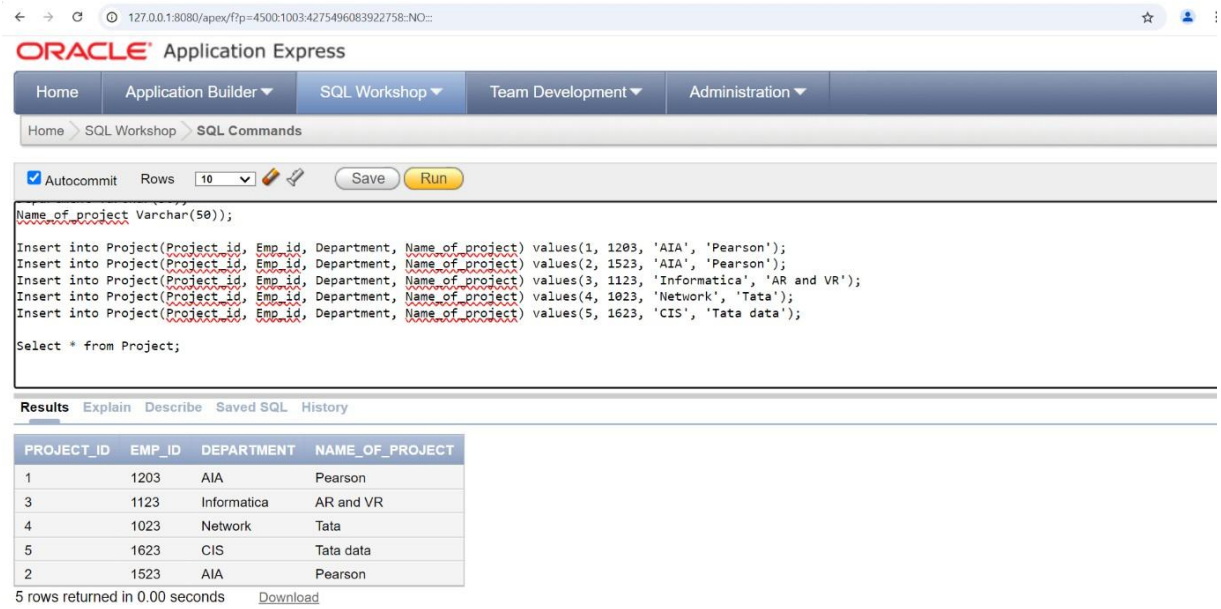
Insert into Project(Project_id, Emp_id, Department, Name_of_project) values(4, 1023, 'Network', 'Tata');

Insert into Project(Project_id, Emp_id, Department, Name_of_project) values(5, 1623, 'CIS', 'Tata data');

Select * from Project;

Output:-

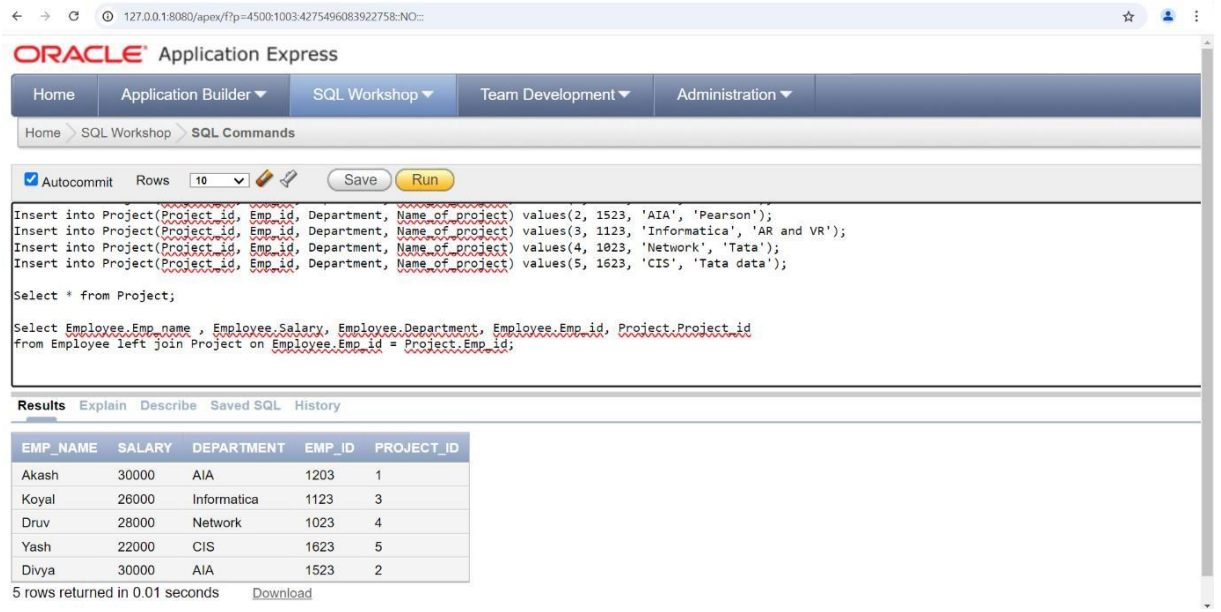




Input:-

Select Employee.Emp_name , Employee.Salary, Employee.Department, Project.Project_id from Employee left join Project on Employee.Emp_id = Project.Emp_id;

Output:-

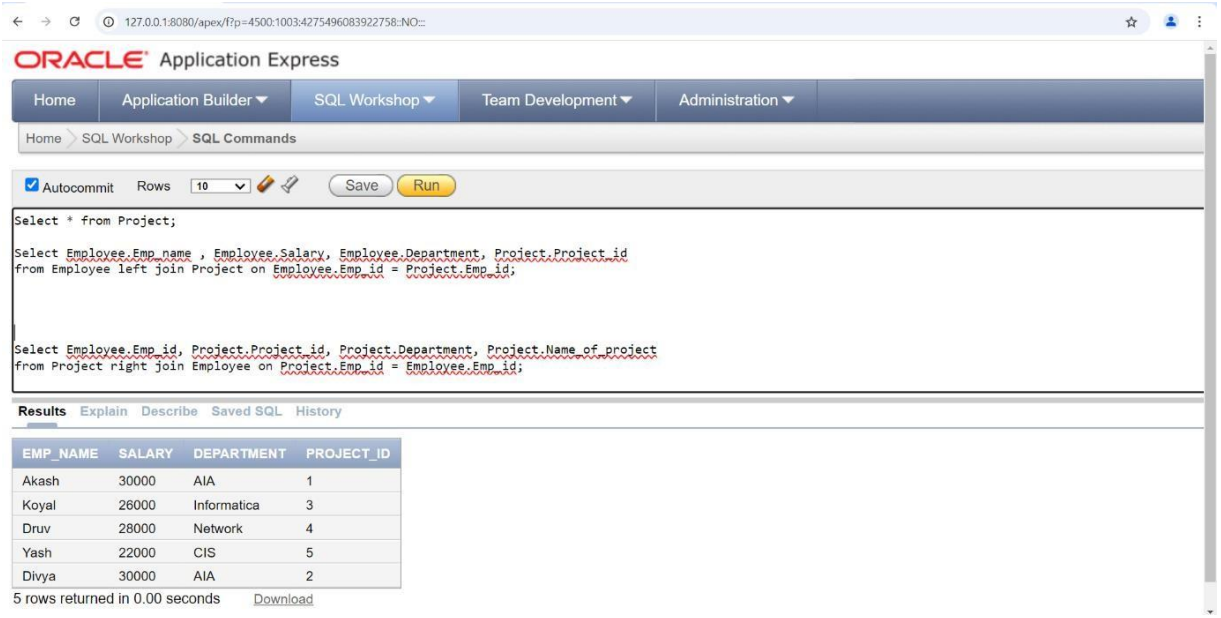


Input:-

Select Employee.Emp_id, Project.Project_id, Project.Department, Project.Name_of_project

from Project right join Employee on Project.Emp_id = Employee.Emp_id;

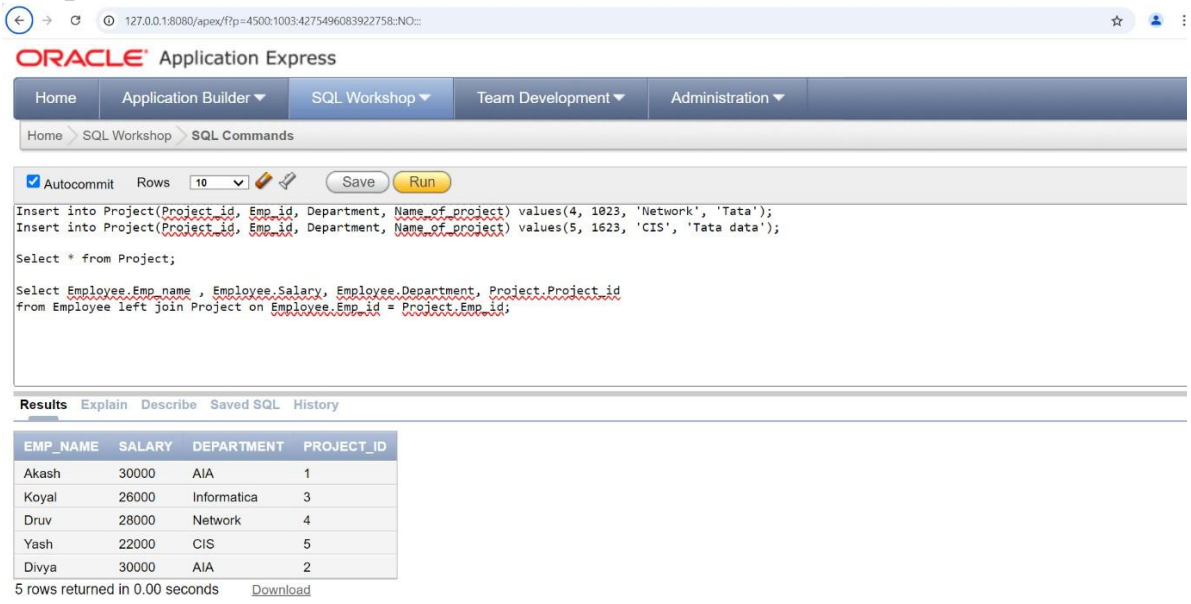
Output:-



Input:-

Select * from Employee full join Project on Employee.Emp_id = Project. Emp_id;

Output:-



Assignment No.5

Aim:- Write a sql statement for implementing DDL Commands.

Apply ddl commands on doctor table

1)Create

2)Drop

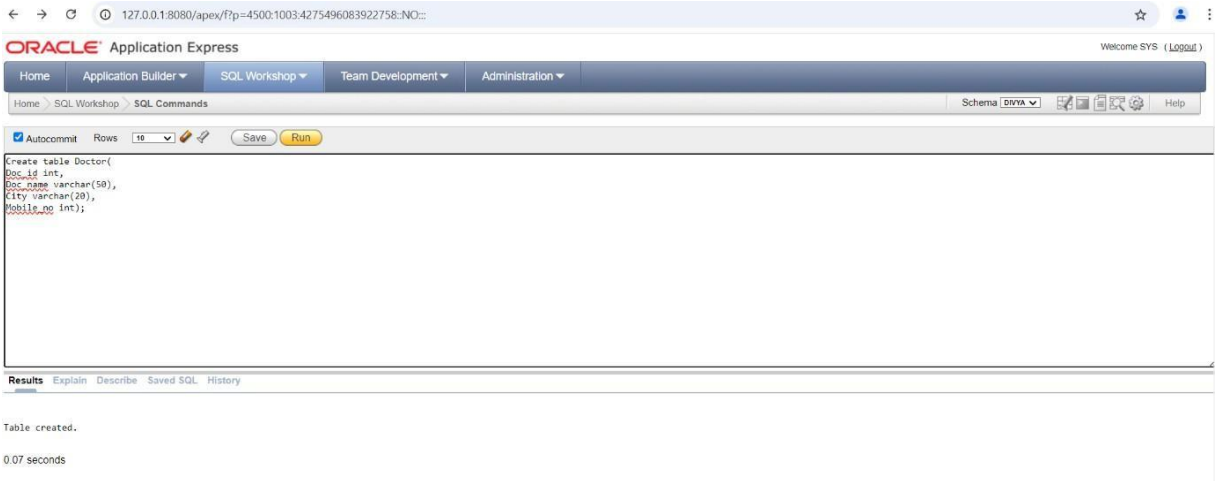
3)Alter

4)Truncate

Input:-

```
Create table Doctor(
Doc_id int,
Doc_name varchar(50),
City varchar(20),
Mobile_no int);
```

Output:-



Input:-

```
Insert into Doctor(Doc_id, Doc_name, City, Mobile_no) values(1, 'Divya', 'Satara', 9635124786);

Insert into Doctor(Doc_id, Doc_name, City, Mobile_no) values(2, 'Kriti', 'Mumbai', 9925741451);
```

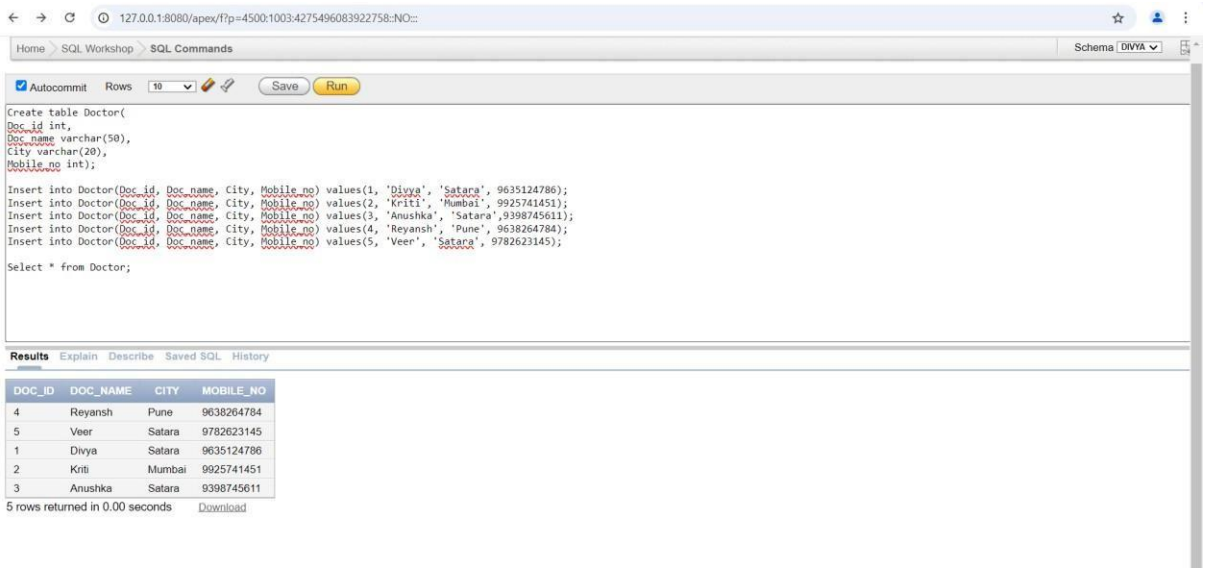
Insert into Doctor(Doc_id, Doc_name, City, Mobile_no) values(3, 'Anushka', 'Satara',9398745611);

Insert into Doctor(Doc_id, Doc_name, City, Mobile_no) values(4, 'Reyansh', 'Pune', 9638264784);

Insert into Doctor(Doc_id, Doc_name, City, Mobile_no) values(5, 'Veer', 'Satara', 9782623145);

Select * from Doctor;

Output:-

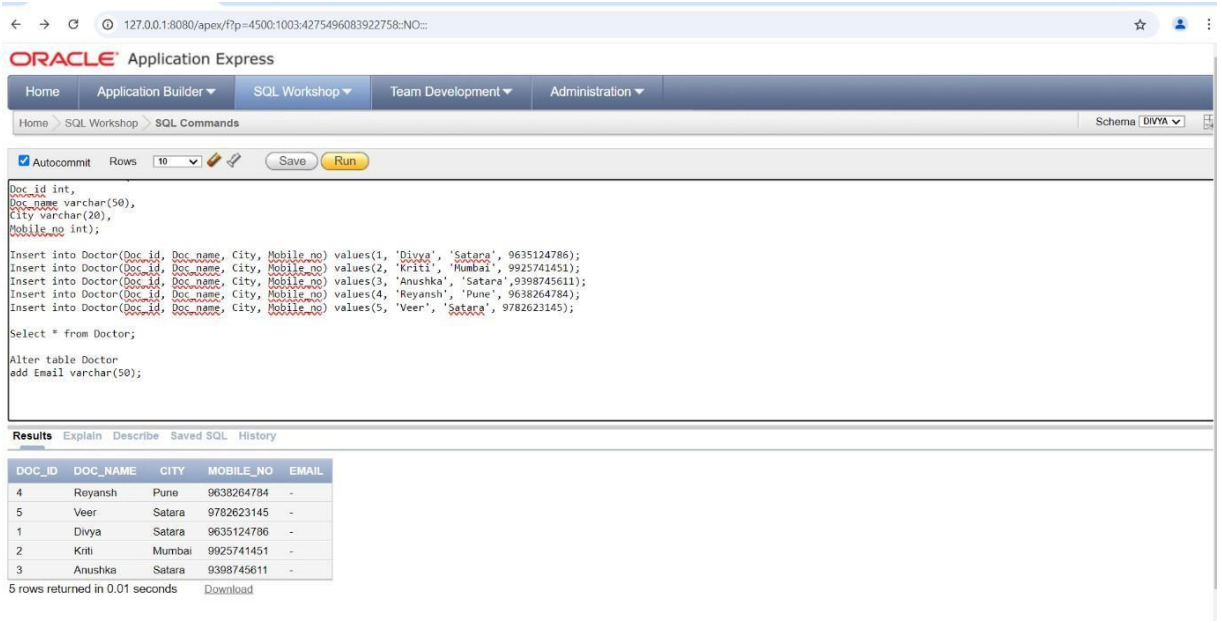


Input:-

Alter table Doctor

add Email varchar(50);

Output:-

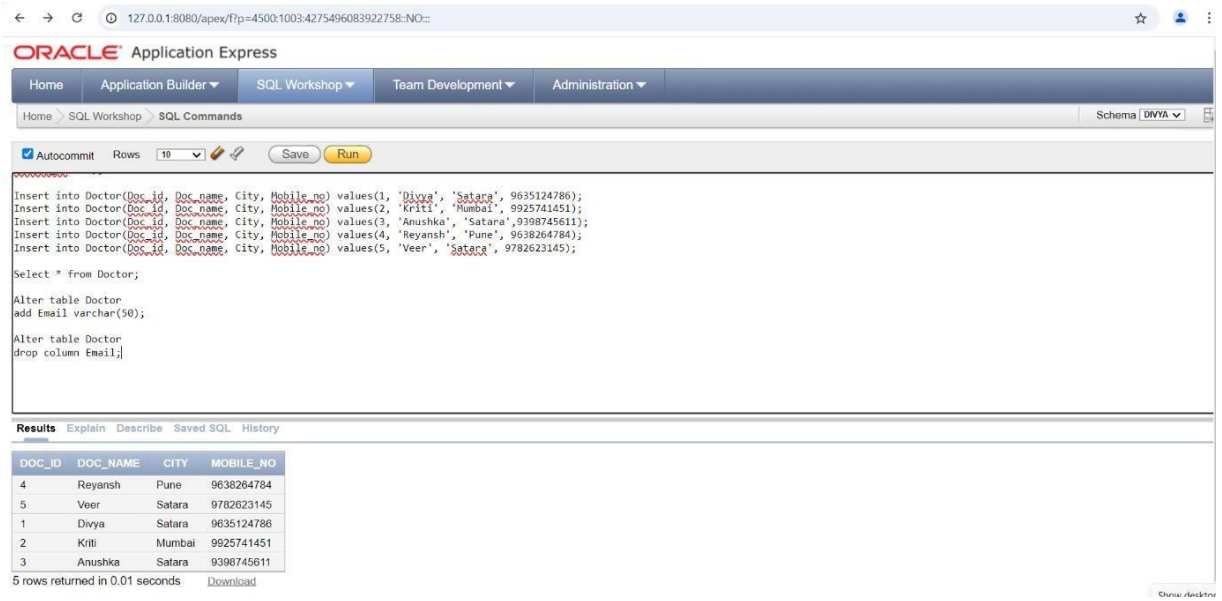


Input:-

Alter table Doctor

drop column Email;

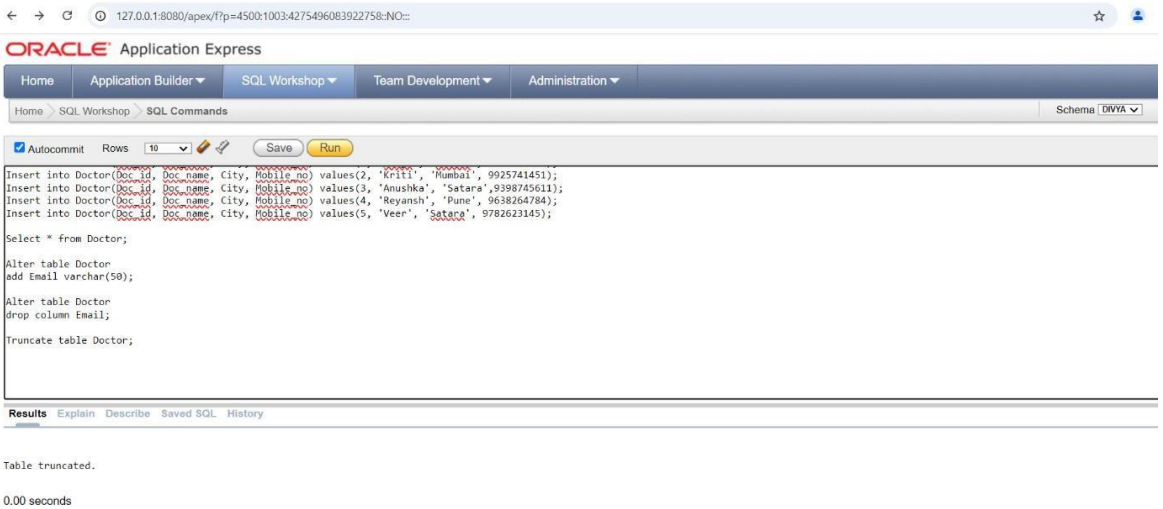
Output:-



Input:-

Truncate table Doctor;

Output:-


127.0.0.1:8080/apex/f?p=4500:1003:42754960839227 58::NO:::
ORACLE Application Express
Home
Application Builder
SQL Workshop
Team Development
Administration
Home SQL Workshop SQL Commands
Schema DIVYA
Autocommit
Rows 10
Save
Run
Insert into Doctor(Doc_id, Doc_name, City, Mobile_no) values(2, 'Kriti', 'Mumbai', 9925741451);
Insert into Doctor(Doc_id, Doc_name, City, Mobile_no) values(3, 'Anushka', 'Satara',9398745611);
Insert into Doctor(Doc_id, Doc_name, City, Mobile_no) values(4, 'Reyansh', 'Pune', 9638264784);
Insert into Doctor(Doc_id, Doc_name, City, Mobile_no) values(5, 'Veer', 'Satara', 9782623145);
Select * from Doctor;
Alter table Doctor
add Email varchar(50);
Alter table Doctor
drop column Email;
Truncate table Doctor;
Results Explain Describe Saved SQL History
Table truncated.
0.00 seconds

Practical No.6

Aim: Perform Transaction Management and Concurrency Control: BEGIN, COMMIT, ROLLBACK.

1)BEGIN Statement

Step1:

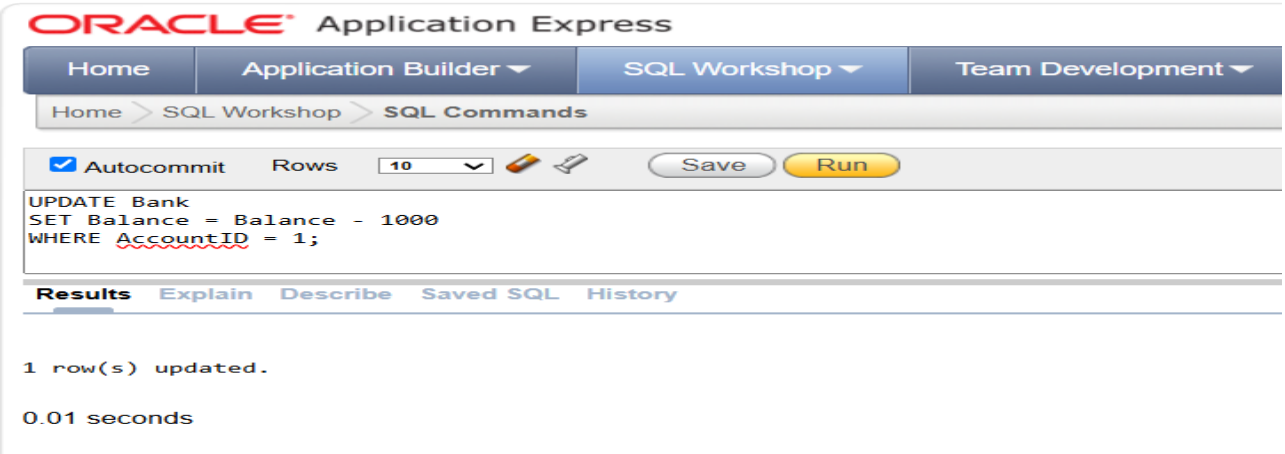
UPDATE Bank SET Balance = Balance - 1000 WHERE AccountID = 1;

Step2:

SELECT * FROM Bank

Query: UPDATE Bank SET Balance = Balance - 1000 WHERE AccountID = 1;

Output:



2) COMMIT Statement

Step1:

UPDATE Bank SET Balance = Balance - 1000 WHERE AccountID = 1

Step2:

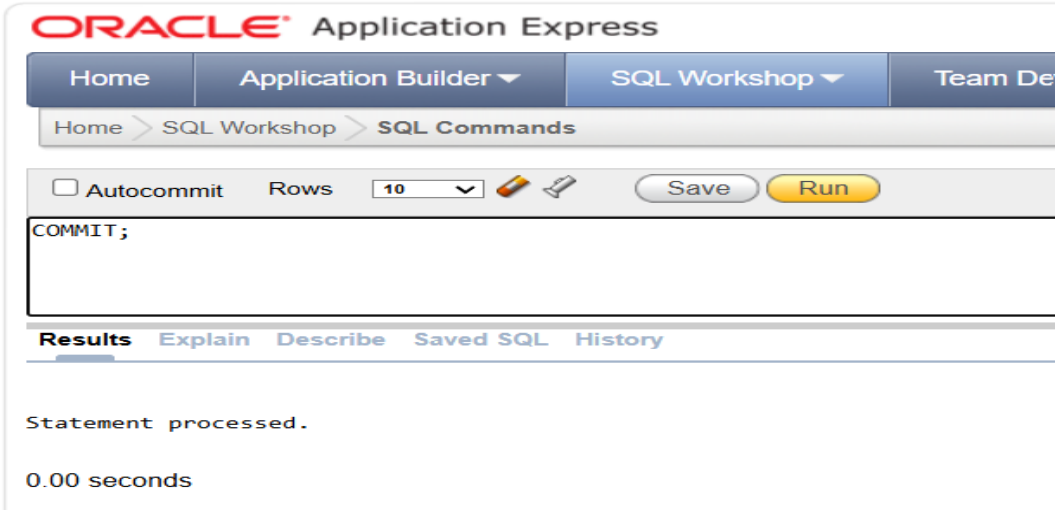
COMMIT

Step3:

SELECT * FROM Bank

Query: COMMIT;

Output:



3)ROLLBACK Statement

Step1:

UPDATE Bank

SET Balance = Balance - 2000

WHERE AccountID = 2;

Step2:

ROLLBACK;

Step3:

SELECT * FROM Bank

Query: ROLLBACK;

Output:

